Лекция

09 октября 2020

3. Функции. Передача аргументов по значению

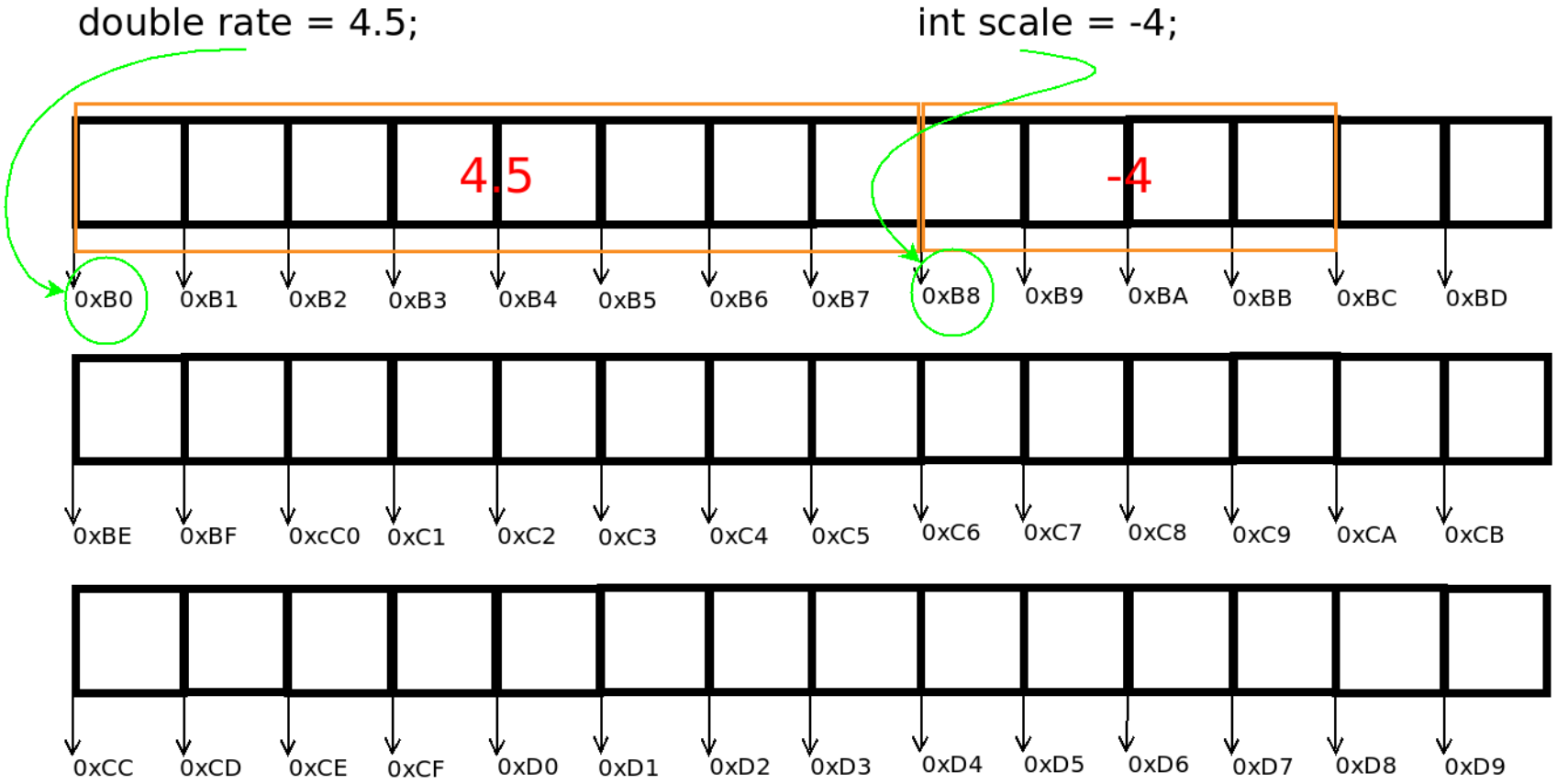
```
1 double degree_nth(double num, int n)
2 { ... }
3
4 double rate = 4.5;
5 int scale = -4;
6
7 double rate_in_scale = degree_nth( rate, ↩
    scale );
```

Прежде, чем идти дальше

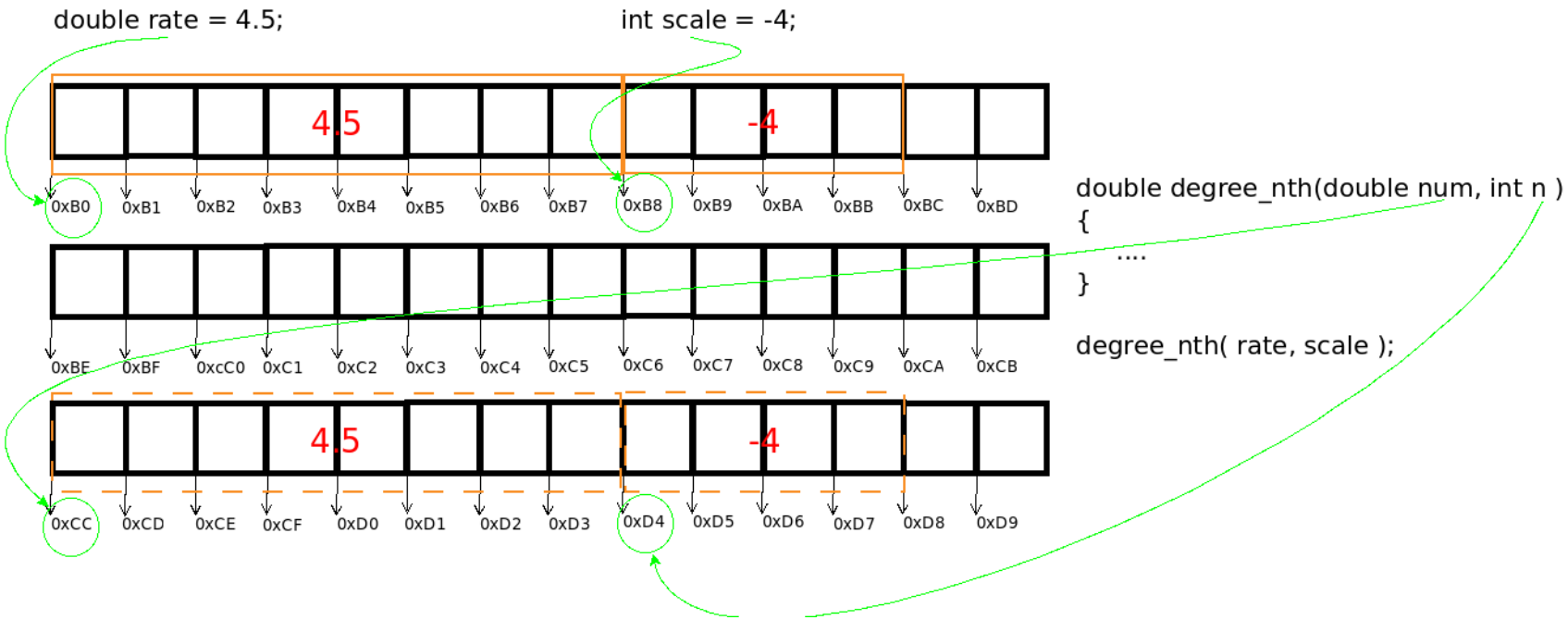
Адрес переменной

Каждая переменная любого типа связана с блоком в памяти, который состоит из какого-либо количества байт. Каждый байт в модели памяти языка C пронумерован - ему присвоен порядковый номер: 0, 1, 2, 3, ... Номер первого байта блока называется **адресом переменной**. Адреса в литературе приводят в *шестнадцатиричном* виде.

3. Функции. Передача аргументов по значению



3. Функции. Передача аргументов по значению



3. Функции. Специальный тип void

Функция может не возращать никакого значения. Для этого в качестве возращаемого типа используется специальный тип - **void**. Вызов оператора **return** не обязателен.

```
1 void print_pair(double first, double second)
2 {
3   printf("<< %lf, %lf >>", first, second);
4
5   // В тоже время, нельзя создать переменную↩
        типа void
6   // void v_var; // Ошибка компиляции!
7 }
```

3. Функции. Специальный тип void

Однако, оператор **return** может присутствовать для немедленного выхода из подобной функции (тип возращаемого значения - **void**).

```
1  void declarate_party(size_t alcohol_litres)
2  {
3    if ( alcohol_litres == 0 ) {
4      return;
5    }
6
7    if ( alcohol_litres < 10 ) {
8      printf("Party good: :)\n");
9      return;
10   }
11
12   printf("Party very good: :D!\n");
13 }
```

3. Функции. Объявление и определение

Аналогично переменным, для функций существуют понятия **объявления** и **определения**. Определение - это все примеры выше, а под **объявлением** понимается указание *типа возращаемого значения, названия функции и перечисление типов всех аргументов*. Причём, давать имена аргументам - не обязательно

```
1  #include <stdio.h>
2
3  double my_random(unsigned); // Объявление!
4
5  int main()
6  {
7    printf("Случайное число: %lf\n", my_random(567)↩
       );
8  }
9
10 double my_random(unsigned seed) // Определение!
11 { return 4.5688 / seed; }
```

Прежде, чем идти дальше

Область видимости идентификатора

Место в файле с исходным кодом, где идентификатор доступен для операций (использование значения в случае переменной, вызов - в случае функции). В языке C различают две области - **глобальная** и **локальная**.

- Все функции имеют **глобальную** область видимости
- Переменная имеет **локальную** область видимости, если её объявление или определение находится внутри пары фигурных скобок (**{}**)
- Иначе переменная получает **глобальную** область видимости
- Локальная переменная видна во всех вложенных областях

Прежде, чем идти дальше

Область видимости идентификатора

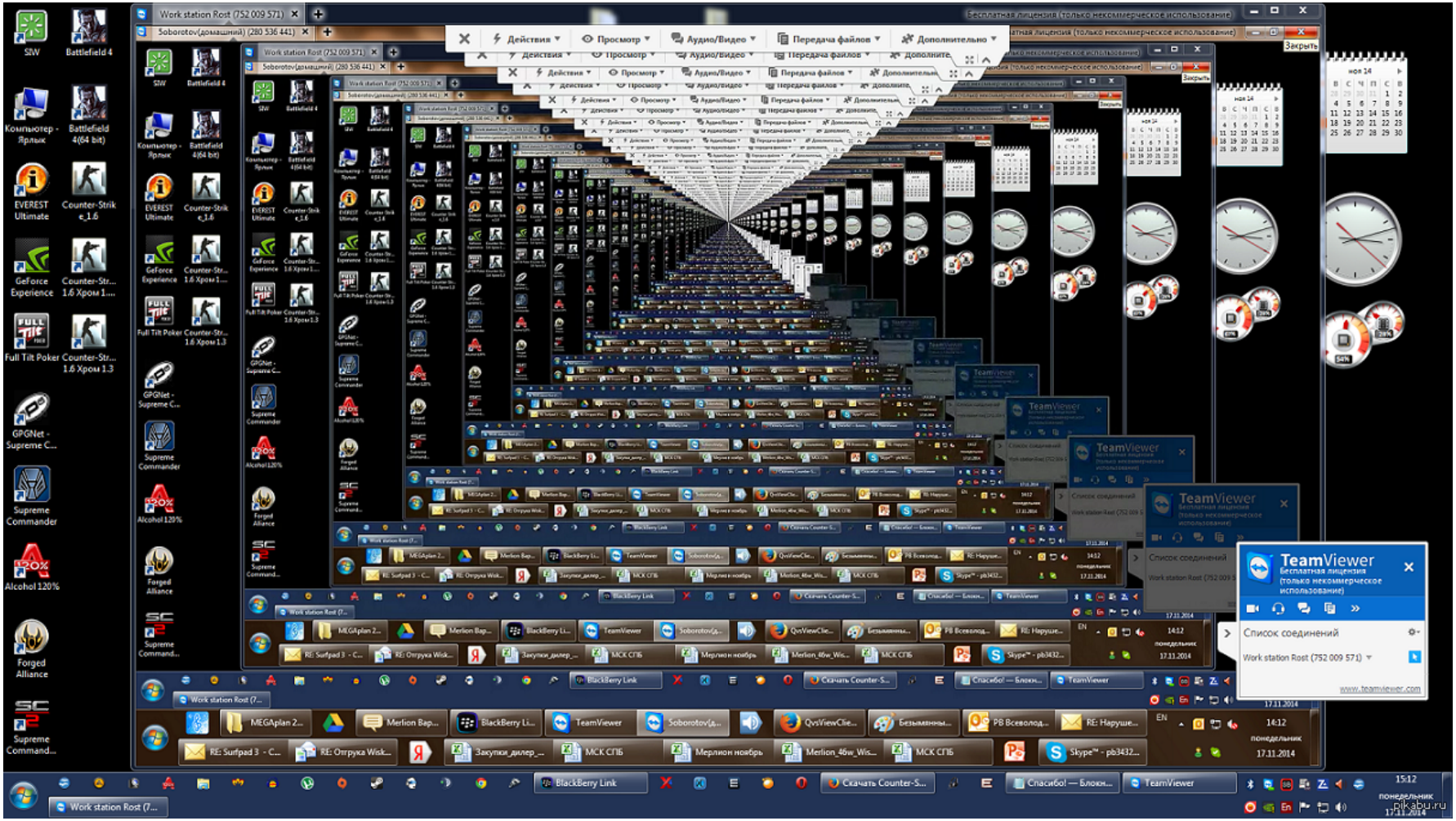
```
1  // Переменная с глобальной областью видимости
2  const double PI = 3.14159265358;
3
4  void do_something(double x)
5  {
6    // Локальная область видимости
7    double rate = 0.5;
8
9    if (x > 10.0) {
10     // Ещё одна. PI и rate тут доступны
11     double num = PI * x * rate;
12     printf("Результат: %lf", num);
13   }
14
15   // Доступа к num уже нет:
16   // num += 3.05;
17   rate *= 9.5;
18 }
```

Прежде, чем идти дальше

Время жизни переменной

- Переменные с локальной областью видимости автоматически удаляются по достижении конца области видимости
- Глобальные переменные существуют до тех пор, пока выполняется программа

Функции. Рекурсия



Функции. Рекурсия

В информатике под *рекурсией* понимается метод, в котором решение исходной задачи зависит от набора решений частных случаев этой же задачи. С для реализации подобных методов предоставляет возможность внутри функции делать вызов самой себя. Такие функции называются **рекурсивными**.

```
1 // Пример бесконечной (и хвостовой) рекурсии
2 int call_self(int num)
3 {
4   printf("Передано число: %d\n", num);
5   return call_self(num);
6 }
```

Функции. Рекурсия

Различают *прямую* и *косвенную* рекурсивную функцию.

- **прямая рекурсивная функция** - вызывает саму себя непосредственно в своём теле
- **косвенная рекурсивная функция** - вызывает некоторую другую функцию, в теле которой происходит обратный вызов исходной функции

```
1  int start(int);
2
3  int process_number(int num)
4  {
5    return start(num);
6  }
7
8  int start(int n)
9  {
10   return process_number(n - 5);
11 }
```

Для того, чтобы быть полезной, рекурсивная функция должна включать два необходимые особенности реализации:

1. Тело функции должно обязательно содержать **условия остановки рекурсивных вызовов**.
2. Параметры функции при рекурсивном вызове **должны меняться, а не оставаться постояными**.

Числа Фибоначчи

$$F_1 = 1, F_2 = 1, F_n(n > 2) = F_{n-1} + F_{n-2}$$

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, ...

Числа Фибоначчи

$$F_0 = 0, F_1 = 1, F_2 = 1, F_n(n \geq 2) = F_{n-1} + F_{n-2}$$

```
1  unsigned long long fib_num(unsigned n)
2  {
3    // условие выхода из рекурсии
4    if (n < 2) {
5      return n;
6    }
7
8    return fib_num(n - 1) + fib_num(n - 2);
9  }
10
11 printf("8-ое число Фибоначи: %llu\n",
12         fib_num(8));
```

Функции. Рекурсия

Рекурсия приводит к повторному выполнению содержащихся в ней инструкций, что аналогично работе цикла.

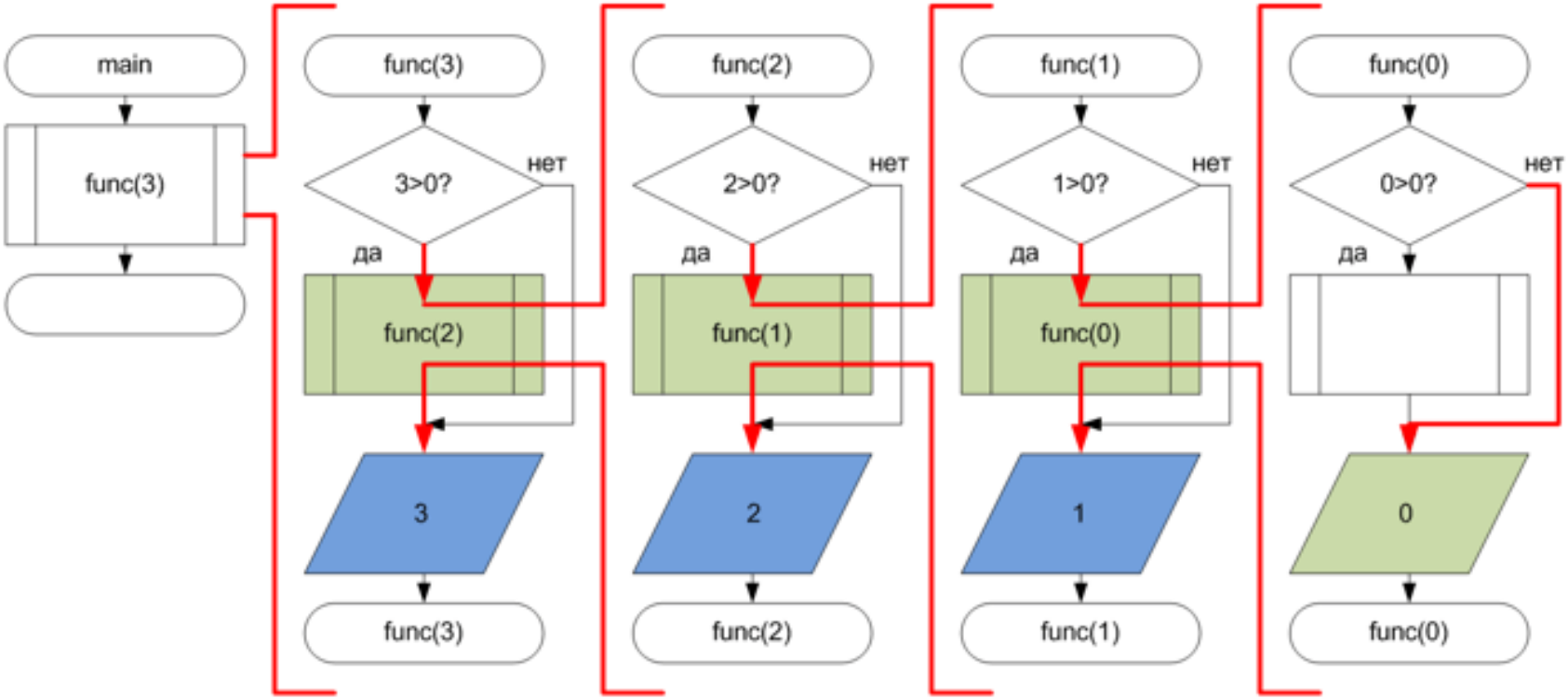
Префиксная форма Сначала - рекурсивный вызов, потом - действия

```
1 void func(int num)
2 {
3   if (num > 0) func(num - 1);
4  printf("%d",num);
5 }
```

Постфиксная форма Сначала - действия, потом - рекурсивный вызов

```
1 void func(int num)
2 {  printf("%d",num);
3    if (num > 0) func(num - 1);
4 }
```

Функции. Рекурсия



Массивы в С

Массив - структура данных, содержащая набор проиндексированных элементов. В языке С массивы являются типизированными, то есть могут содержать элементы только одного типа.

```
<тип> <имя_массива>[<размер>];
```

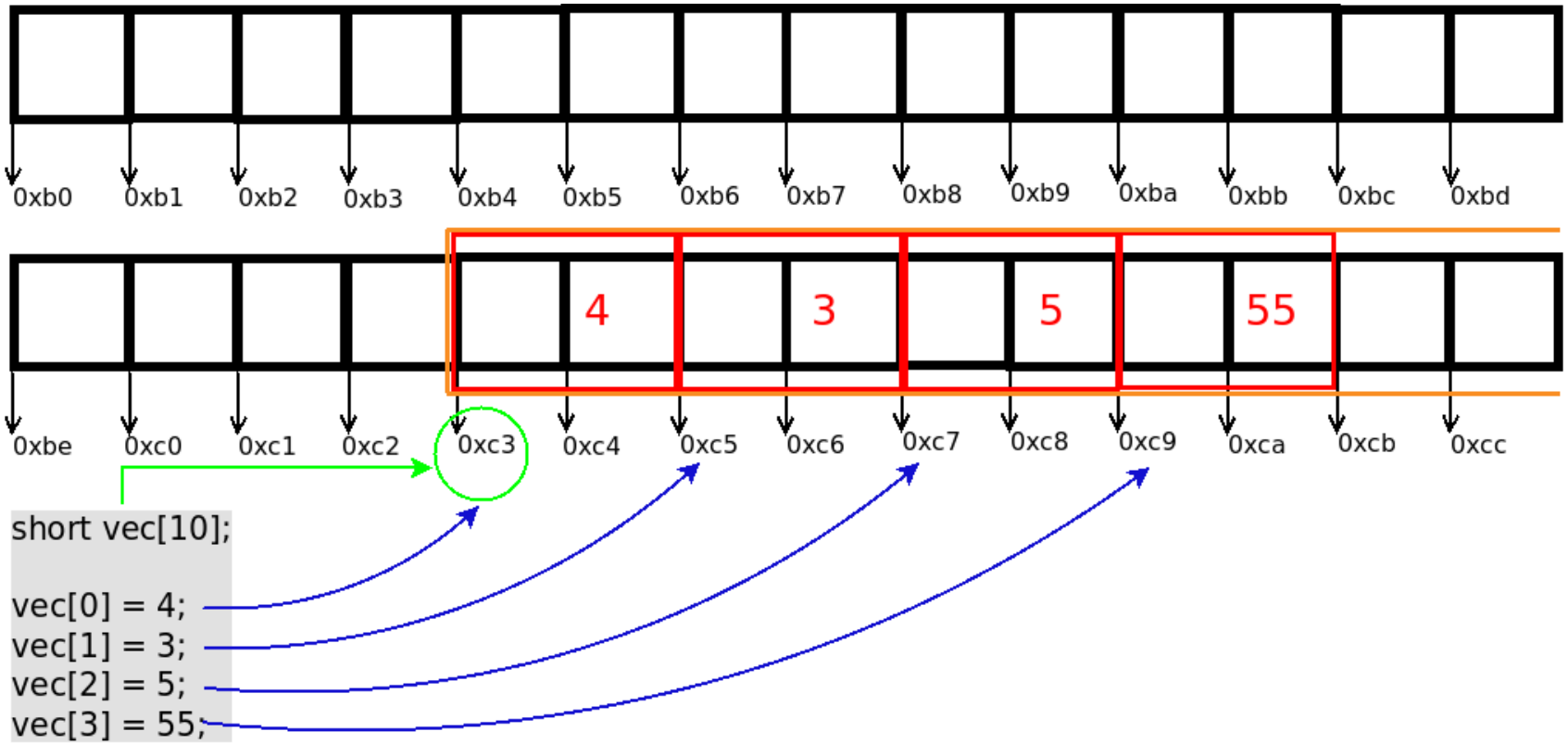
В С все элементы одного массива располагаются в памяти последовательно.

Статический массив

Индексация элементов в массиве **начинается с нуля**.

```
1 short vec[10];
2
3 // Задаём значение первого элемента
4 vec[0] = 4;
5 vec[3] = 55;
6
7 printf("Первый элемент равен %d", vec[0]);
```

Точка зрения памяти



Инициализация массива - задание начальных значений каждому элементу

```
1  int vec[8] = { 1, 2, 3, 4, 5, 6, 7, 8 };
2
3  for (unsigned i = 0; i < 8; ++i) {
4    printf("%u ", vec[i]);
5  }
6  printf("\n");
7
8  // Незаданные элементы будут равны 0.0
9  double real_arr[5] = { 3.4, 5.5, 77.11 };
10
11 // При явной инициализации размер массива ↩
     можно пропускать
12 int another_vec[] = { 1, 2, 3, 4 };
```

Массивы в С

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #define SIZE 20
4 int main () {
5  int a[SIZE];int n;
6   // Заполнение массива случайными числами
7   for (int i = 0; i<SIZE; i++)
8     a[i] = rand() % 201 - 100;
9   // Ввод элементов массива
10  scanf("%d", &n);
11  for (i = 0; i<n; i++)
12  {
13    printf("a[%d] = ", i);
14    scanf("%d", &a[i]);
15  }
16 }
```

Двумерные массивы(матрицы).

$$A[i][j]$$

$$\begin{array}{cc} & \xrightarrow{\quad j \quad} \\ i\downarrow & \begin{matrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{m,1} & \cdots & A_{m,n} \end{matrix} \end{array}$$

Массивы в С

```
1  int matrix1[10][10];
2
3  for (int i = 0; i < 10; ++i) {
4    for (int j = 0; j < 10; ++j) {
5      matrix[i][j] = i + j;
6    }
7  }
8  // Инициализация
9  int matrix2[3][3] = { {1, 2, 3}, {4, 5, 6}, ↩
     {7, 8, 9} };
10 //Вывод
11 for (int i = 0; i < 10; ++i) {
12   for (int j = 0; j < 10; ++j) {
13     printf("%5d", matrix[i][j])
14   }
15   printf("\n");
16 }
```

Многомерные массивы: в числе размерностей никто не ограничен. Гарантируется работа до 31 уровня вложенности.

```
1 int monstr[3][4][5][3][4][5];
2 monstr[0][0][0][0][0][0] = 5;
```

Передача массивов в функции.

```
1  void print_array(int arr[], size_t count)
2  {
3    printf("\nПереданный массив:\n");
4    for (size_t i = 0; i < count; ++i) {
5      printf("%d ", arr[i]);
6    }
7    printf("\n");
8  }
9
10 ...
11
12 int vec[8] = { 1, 2, 3, 4, 5, 6, 7, 8 };
13 print_array(vec, 8);
```

Никакой проверки размерности массива не происходит.

```
1  void print_array(int arr[55], size_t count)
2  {
3    printf("\nПереданный массив:\n");
4    for (size_t i = 0; i < count; ++i) {
5      printf("%d ", arr[i]);
6    }
7    printf("\n");
8  }
9
10 ...
11
12 int vec[8] = { 1, 2, 3, 4, 5, 6, 7, 8 };
13 print_array(vec, 8);
```

Многомерные массивы: нужно указать все размерности, кроме первой.

```
1  const size_t COLS = 8;
2
3  void print_2D_array(int matr[][COLS], size_t↩
       rows_count)
4  {
5    printf("\nПереданный двумерный массив:\n")↩
       ;
6    for (size_t i = 0; i < rows_count; ++i) {
7      for (size_t j = 0; j < COLS; ++j) {
8        printf("%d ", matr[i][j]);
9      }
10     printf("\n");
11   }
12   printf("\n");
13 }
```

Сортировка Массивов

Сортировка — упорядочение (перестановка) элементов в подмножестве данных по какому-либо критерию. Чаще всего в качестве критерия используется некоторое числовое поле, называемое ключевым. Упорядочение элементов по ключевому полю предполагает, что ключевое поле каждого следующего элемента не больше предыдущего (**сортировка по убыванию**). Если ключевое поле каждого последующего элемента не меньше предыдущего, то говорят о **сортировке по возрастанию**.

Методы сортировки массивов:

- сортировка обменом
- сортировка вставками;
- сортировка слиянием;

Линейная сортировка

```
1   for (int i = 0; i < size - 1; i++)
2   {
3     min = i; // запоминаем индекс текущего
4     // ищем минимальный элемент
5     for (int j = i + 1; j < size; j++)
6     {
7       if (num[j] < num[min])
8         min = j; // запоминаем его индекс
9     }
10 // меняем местами i-ый и минимальный ↩
     элементы
11    temp = num[i];
12    num[i] = num[min];
13    num[min] = temp;
14  }
```

Метод «пузырька»

```
1  #include<stdio.h>
2  #define N 1000
3 int main() {int a[N];
4   int n, i, j;
5   scanf("%d", &n); //считываем размер
6   for(i = 0 ; i < n; i++) {
7     scanf("%d", &a[i]); }
8   for(i = 0 ; i < n - 1; i++) {
9     for(j = 0 ; j < n - i - 1 ; j++) {
10      if(a[j] > a[j+1]) {// сравниваем
11        int tmp = a[j];// если нетак
12        a[j] = a[j+1] ;//  меняем
13        a[j+1] = tmp;
14      }
15    }
16  }
17 }
```

Метод «пузырька» вариант 2

```
1  int obmen=1;i = 0
2      while( obmen&&(i < n - 1)) {
3          obmen=0;
4          for(j = 0 ; j < n - i - 1 ; j++) {
5              if(a[j] > a[j+1]) {
6                  obmen=1;
7                  int tmp = a[j];
8                  a[j] = a[j+1] ;
9                  a[j+1] = tmp;
10             }
11         }
12     i++;
13     }
```

Быстрая сортировка (метод Хоара)

```
1 void quickSort(int numbers[ ], int left, int↩
    right)
2 {
3 int pivot; // разрешающий элемент
4 int l_hold = left; //левая граница
5 int r_hold = right; // правая граница
6 pivot = numbers[left];
7 while (left < right) // пока границы не ↩
    сомкнутся
8 {while ((numbers[right] >= pivot) && (left <↩
    right))
9   right--; // сдвигаем правую границу
10  if (left != right) // если границы не ↩
    сомкнулись
11  {numbers[left] = numbers[right];
12   left++; // сдвигаем левую границу вправо
13  }
```

Быстрая сортировка (метод Хоара)

```
1  while ((numbers[left] <= pivot) && (left < right))
2   left++; // сдвигаем левую границу
3     if (left != right) // если границы не сомкнулись
4     {
5       numbers[right] = numbers[left];
6       // перемещаем элемент [left] на место [right]
7       right--; // сдвигаем правую границу вправо
8     }
9   }
10  numbers[left] = pivot;
11  // ставим разрешающий элемент на место
```

Быстрая сортировка (метод Хоара)

```
1 // Рекурсивно вызываем сортировку для левой ↩
    и правой части массива
2  pivot = left;
3   left = l_hold;
4   right = r_hold;
5   if (left < pivot)
6     quickSort(numbers, left, pivot - 1);
7   if (right > pivot)
8     quickSort(numbers, pivot + 1, right);
9 }
```

Другие сортировки

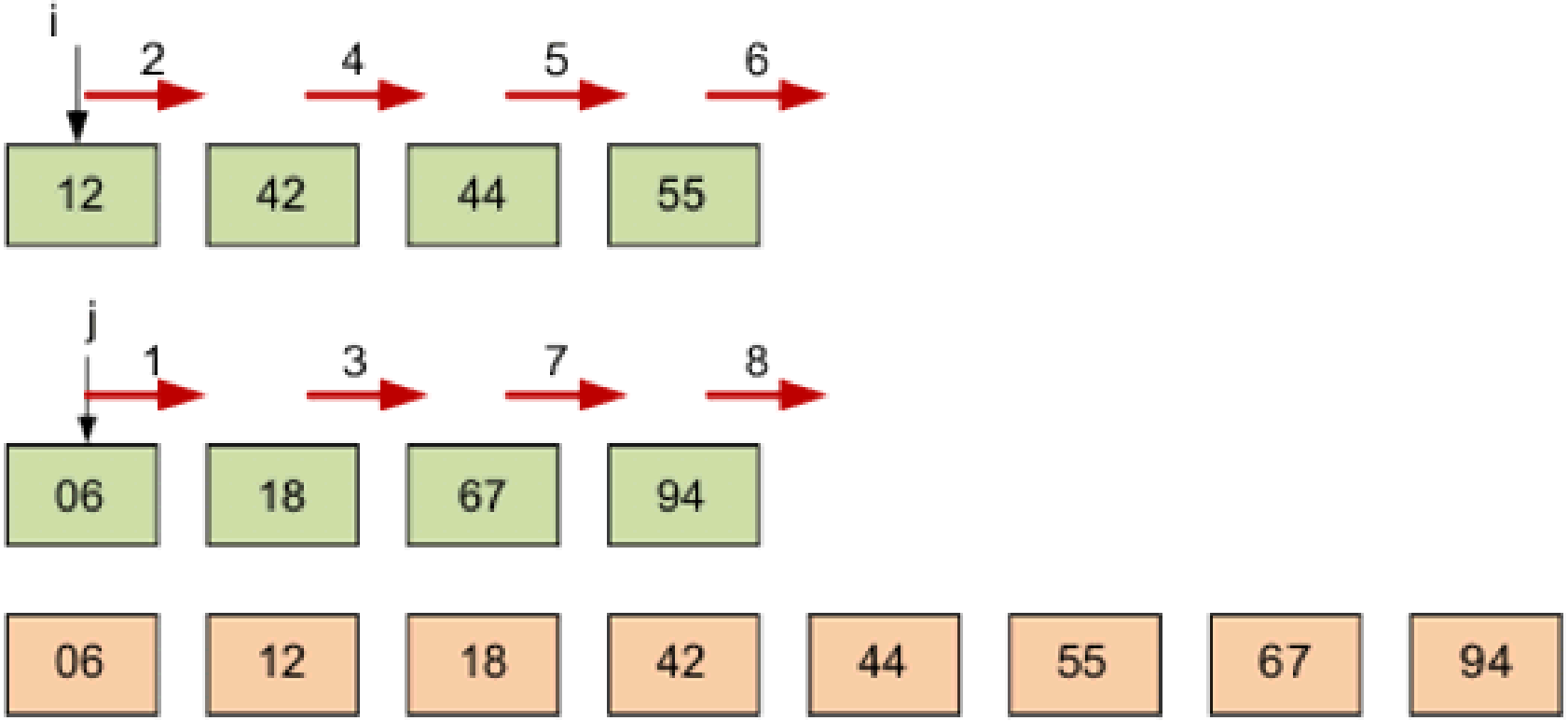
- Челночная сортировка
- Сортировка Шелла
  1,4,13,40,121. . . , где $h_{k-1} = 3h_k + 1; h_p = 1p = log_2(n) - 1;$
  1,3,7,15,31,. . . , где $h_{k-1} = 2h_k + 1; h_p = 1p = log_2(n) - 1;$
- Сортировка вставками
  Линейная вставка
  Двоичная вставка
- Сортировка слиянием
  T(n)=2T(n/2)+O(n)=4T(n/4)+2O(n/2)+O(n)=4T(n/4)+2O(n)=
  . . .
  = 2kT(1)+kO(n) = O(n)+log2nO(n)=O(nlog2n)
- Сортировка бинарным деревом
  i-й элемент родитель 2*i, 2*i+1 потомки
  $O(nlog_2n)$ – min для сортировок, использующих попарное сравнение

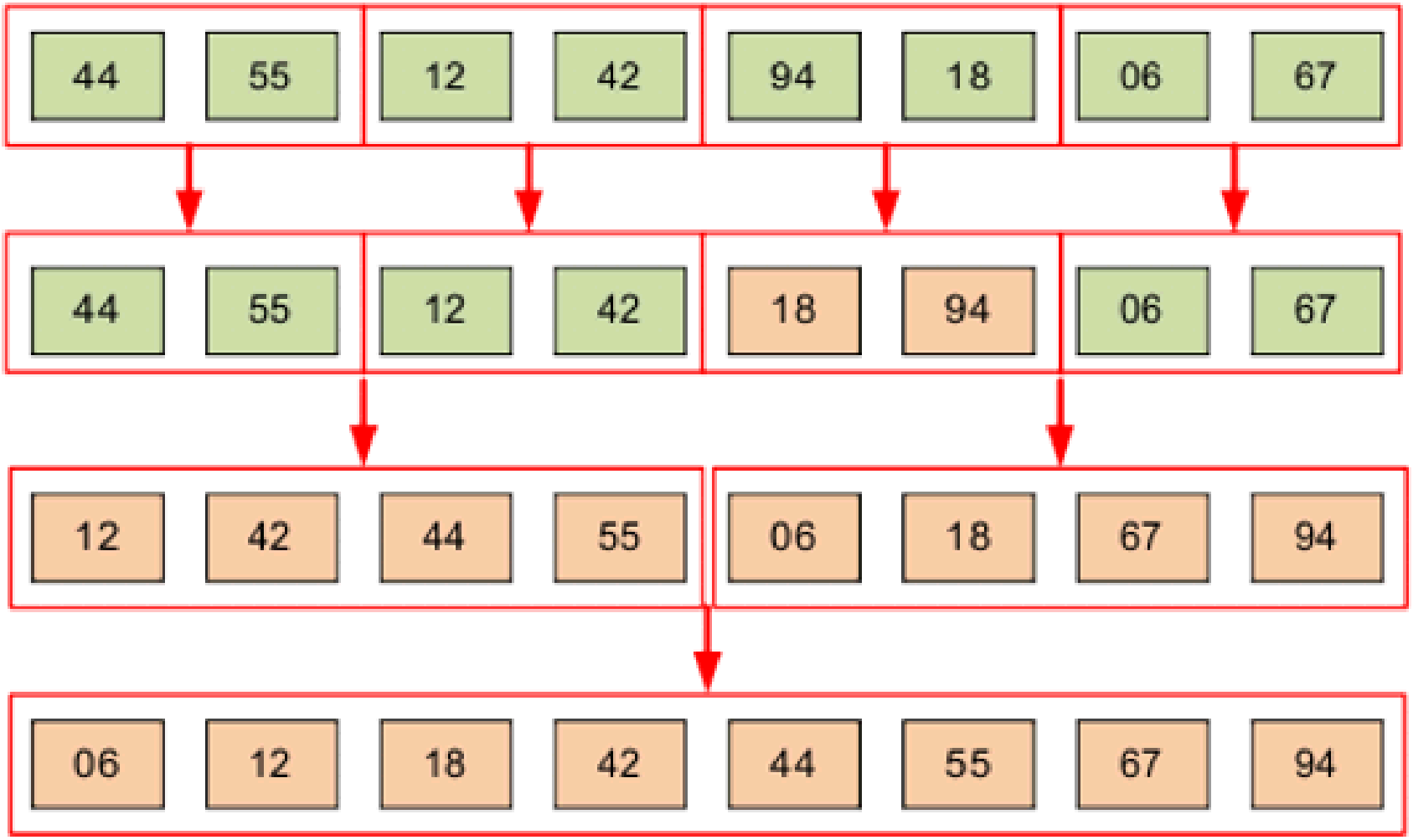
Сортировка:Слиянием

Сортировка слиянием — алгоритм сортировки, который упорядочивает списки (или другие структуры данных, доступ к элементам которых можно получать только последовательно, например — потоки) в определённом порядке.
Основной применяемый метод для сортировки файлов — сортировка слиянием.
Недостатки метода: требует дополнительной памяти по объему равной объему сортируемого файла.

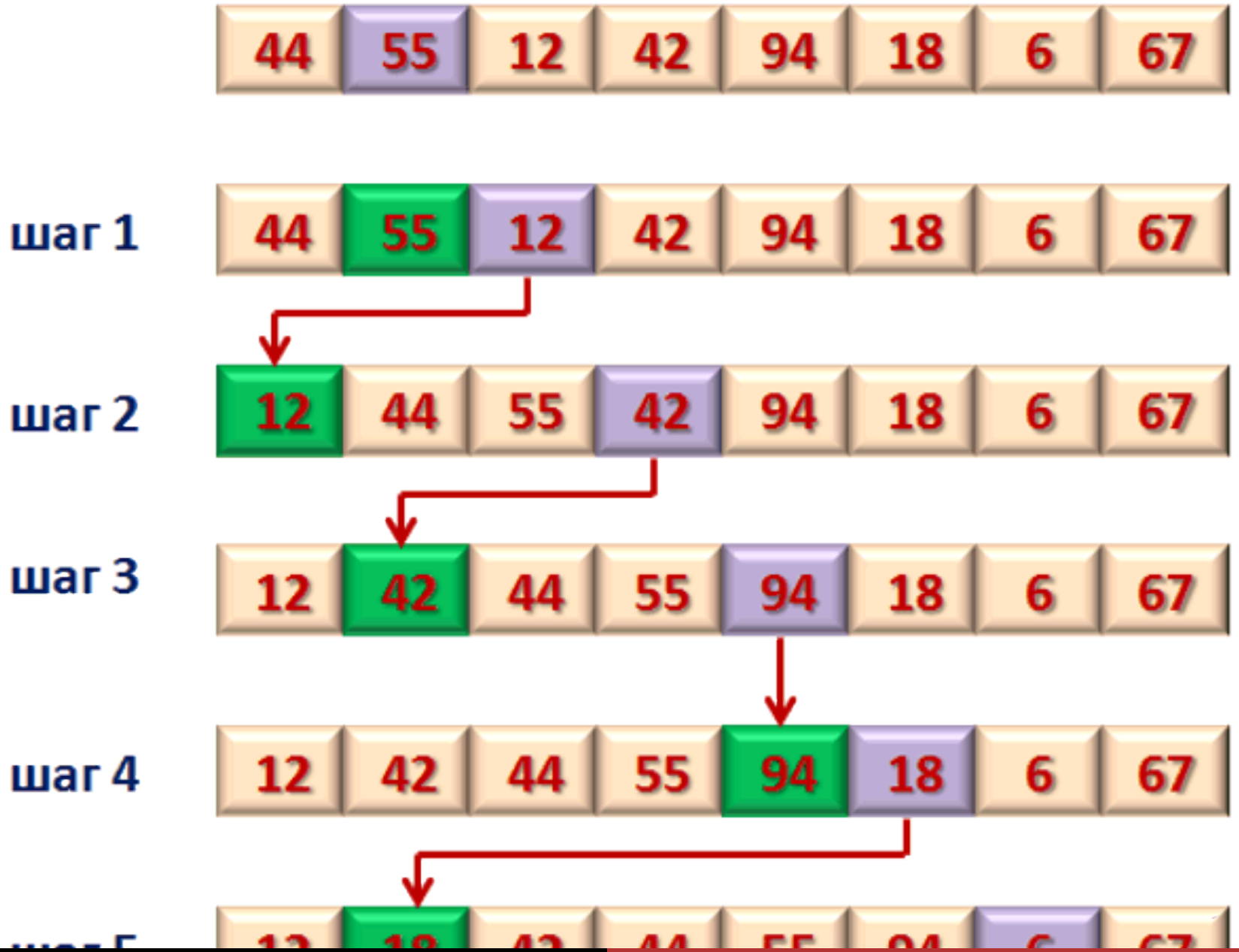
Сортировка:Слиянием

Начальные элементы предварительно упорядоченных последовательностей сравниваются между собой, и из них выбирается наименьший.
Соответствующий указатель перемещается на следующий элемент.
Процедура повторяется до тех пор, пока не достигнут конец одной из подпоследовательностей.
Оставшиеся элементы другой подпоследовательности при этом передаются в результирующую последовательность в неизменном виде.


i
2
4
5
6
12
42
44
55
j
1
3
7
8
06
18
67
94
06
12
18
42
44
55
67
94


44 55 12 42 94 18 06 67
44 55 12 42 18 94 06 67
12 42 44 55 06 18 67 94
06 12 18 42 44 55 67 94

Сортировка:Линейная вставка



Сортировка:Линейная вставка

```
1  void inclusionSort(int num[], int size)
2  {
3  // Для всех элементов кроме начального
4  for (int i = 1; i < size; i++)
5   {
6    int value = num[i];
7    int index = i;
8    while ((index > 0) && (num[index - 1] > ↩
       value))
9    { // смещаем элементы к концу массива
10    num[index] = num[index - 1];
11    index--;
12   }
13   num[index] = value; // Вставка
14  }
15 }
```

Алгоритмы поиска

- Линейный (для неупорядоченного массива единственный)
  Число сравнений в среднем n/2
- Бинарный
  Один шаг уменьшает число проверяемых записей от n до n/2
  Число сравнений в среднем $log_2 n + 1$
- Интерполяционный
  Если a[1]<a<a[r], номер очередного элемента
  m = 1+(r-1)*(a-a[1])/(a[r]-a[1]).
  Один шаг уменьшает число проверяемых записей от n до $\sqrt{n}$
  Число сравнений в среднем $\log_2 \log_2 n$

Бинарный Алгоритм поиска

Один шаг уменьшает число проверяемых записей от n до n/2
Число сравнений в среднем $log_2 n + 1$
На каждом шаге осуществляется поиск середины отрезка по формуле
$mid = (left + right)/2$
Если искомый элемент равен элементу с индексом mid, поиск завершается.
В случае если искомый элемент меньше элемента с индексом mid, на место mid перемещается правая граница рассматриваемого отрезка, в противном случае — левая граница.

Бинарный Алгоритм поиска

```
1  int k[20];
2  printf("Enter key: "); // искомый ключ
3  scanf("%d", &key);
4  int left = 0,right = 19;
5  int search = -1; // элемент не найден
6  while (left <= right) // пока левая граница ↩
       не "перескочит" правую
7  { int mid = (left + right) / 2; //середина
8      if (key == k[mid]) { //ключ найден
9        search = mid;
10       break; }
11  if (key < k[mid]) //ключ меньше  середины
12   right = mid - 1;// смещаем правую границу
13  else
14  left = mid + 1;    // смещаем левую границу
15   }
```